```
re = /
  \A
    (?<brace_expression>
      {
  (
    [^{}]                       # anything other than braces
  |                             # ...or...
    \g<brace_expression>        # a nested brace expression
  )*
      }
    )
  \Z
/x
```

We use the x option to allow us to write the expression with lots of space, which makes it easier to understand. We also indent it, just as we would indent Ruby code. And we can also use Ruby-style comments to document the tricky stuff. You can read this regular expression as follows: a brace expression is an open brace, then a sequence of zero or more characters or brace expressions, and then a closing brace.

Nested Groups

The ability to invoke subpatterns recursively means that backreferences can get tricky. Ruby solves this by letting you refer to a named or numbered group at a particular level of the recursion—add a +n or -n for a capture at the given level relative to the current level.

Here's an example from the Oniguruma cheat sheet. It matches palindromes:

```
/\A(?<a>|.|(?:(?<b>.)\g<a>\k<b+0>))\z/
```

That's pretty hard to read, so let's spread it out:

tut_regexp/palindrome_re.rb

```
palindrome_matcher = /
\A
  (?<palindrome>
                # nothing, or
   | \w         # a single character, or
   | (?:        # x <palindrome> x
       (?<some_letter>\w)
       \g<palindrome>
       \k<some_letter+0>
     )
  )
\z
/x

palindrome_matcher.match "madam" # => madam
palindrome_matcher.match "m"     # => m
palindrome_matcher.match "adam"  # =>
```

A palindrome is an empty string, a string containing a single character, or a character followed by a palindrome, followed by that same character. The notation \k<some_letter+0> means that the letter matched at the end of the inner palindrome will be the same letter that was at the start of it. Inside the nesting, however, a different letter may wrap the interior palindrome.